

Private Authorization Witnessing

A concise positioning document for WitnessKey, LOOPtLOOP, and Abracadoo

Private consent. Publicly auditable proof. Confirmation without custody.

One-sentence position

WitnessKey provides privacy-preserving third-party confirmation of authorization events: participants exchange the sensitive consented payload privately, while WitnessKey witnesses only a hash, consent envelope, role declarations, code/protocol version, timing window, and receipt signature.

What this service is

- **A third-party authorization witness:** It confirms that an authorization event was presented for witness under a declared schema and consent envelope.
- **Hash-first, content-private infrastructure:** The private authorization payload stays with the participants unless they later choose to disclose it.
- **A loop-aware consent receipt layer:** The event is not merely timestamped; it is bounded by roles, scope, consent conditions, expiry, and verification rules.
- **An AI-agent authorization primitive:** One participant can be a user, one can be an AI agent, and future instances can support any noun-to-noun combination: human, agent, app, device, organization, place, or object.

System roles

Component	Role	Boundary
Abracadoo	Participant runtime: accepts loop offers, stores local receipts, preserves path/loop distinctions.	Does not automatically create a Relationship from a witnessed event.
WitnessKey / WitnessMark	Witness surface: hashes content or authorization payloads, requests witness events, displays/verifies receipts.	Does not need custody of private payloads.
LOOPtLOOP	Binding protocol: defines loop event grammar, consent envelope, expiry, participant roles, and verification structure.	Does not need to become the app, wallet, or content repository.

Core event model

Participants privately exchange or agree to an authorization payload. WitnessKey receives only the witnessable envelope:

- payload hash
- event type and schema version
- participant-declared roles
- scope label and allowed actions

- consent prompt hash
- client/auditable-code version or hash
- created-at and expires-at window
- optional participant signatures
- WitnessKey receipt signature and verification URL

The witness proves the authorization envelope was witnessed. It does not need to see the private authorization content.

Receipt claims and non-claims

The receipt may claim	The receipt must not claim
A private authorization payload with hash H was presented for witnessing.	That the private payload is true, legal, wise, or complete.
The event used a declared schema, consent prompt, role declaration, and time window.	That the authorizer had legal capacity or was free from coercion.
WitnessKey signed a receipt over the event envelope.	That the actor complied with the authorization after the event.
A later-disclosed payload can be matched to the witnessed hash.	That WitnessKey saw or stored the private authorization content.

Initial product wedge

Witnessed Authorization Loop - a web-based service where a user and an AI agent, or any two participants, privately form an authorization payload and create a signed third-party receipt for the hash of that payload.

First usable scenario:

1. A user authorizes an AI agent to perform a bounded action.
2. The authorization payload is generated locally and exchanged privately between user and agent.
3. WitnessKey receives only the hash and consent envelope.
4. LOOPtLOOP binds the event into a time-bounded loop with declared roles and expiry.
5. Abracadoo stores the participant-side receipt locally.
6. WitnessKey provides a verification page for the signed hash receipt.

Market language

- Private authorization, witnessed by hash.
- Consent receipts for AI agents and delegated action.
- Auditable proof of authorization without custody of private content.
- Third-party confirmation of consented events, without surveillance.
- Authorization receipts for the agentic web.

Minimum viable service

The minimum viable service is not a full identity platform, agent framework, or legal notary. It is a small, verifiable flow:

- Create private authorization payload locally.

- Hash payload in auditable client code.
- Present clear consent prompt and storage/non-storage disclosure.
- Submit hash-only event envelope to WitnessKey/LOOPtLOOP.
- Receive signed receipt and verification URL.
- Store or export receipt through Abracadoo.

Development sequence

- **1. Spec:** Define WITNESSKEY_PRIVATE_AUTHORIZATION_0.1 and LOOPtLOOP_WITNESS_EVENT_1 schemas.
- **2. WitnessKey stub integration:** Replace the external LOOPtLOOP placeholder with a real hash-only witness-offer call.
- **3. Abracadoo accept route:** Add accept-loop flow for viewing, consenting to, and storing external witness receipts.
- **4. Cloudflare API:** Deploy Workers + D1 endpoints for witness offers, witness events, receipts, and verification.
- **5. Verification page:** Render a public receipt verifier showing claims, non-claims, hash, signature, schema, and expiry.
- **6. Agent authorization demo:** Demonstrate user-to-AI-agent authorization witnessed privately by hash.

Strategic thesis

Abracadoo participates. WitnessKey witnesses. LOOPtLOOP binds.

The first durable service is Private Authorization Witnessing: a consent-aware, hash-only receipt system for AI agents, users, apps, devices, and future loop-capable nouns. It offers auditable third-party confirmation of authorization without demanding custody of the private authorization payload.